Домашнее задание

Абсолютная и относительная погрешность.
Округление и накопление ошибки

Формат сдачи: файл `.py` (или `.ipynb`) с решениями и короткие выводы по каждой задаче (2–3 предложения).

Блок 1. Абсолютная и относительная погрешность (задачи 1–5)

Задача 1. Ручной счёт.

Даны приближённые и точные значения. Для каждой пары вычислите абсолютную погрешность Δx и относительную δx (в %):

1. $x = 3.14$, $x^* = \pi$
2. $x = 2.718$, $x^* = e$
3. $x = 9.8$, $x^* = 9.81$ (ускорение свободного падения)
4. $x = 0.333$, $x^* = 1/3$

Вывод: расположите пары по «качеству» приближения. Совпадает ли порядок по Δx и по δx? Если нет — объясните почему.

Задача 2. Сравнение несопоставимых величин.

Инженер измерил:

- длину моста: $x = 1247$ м при точном 1250 м;
- диаметр болта: $x = 9.7$ мм при точном 10.0 мм.

Вычислите Δx и δx для обоих. Кто ошибся «сильнее»? Почему ответ зависит от того, какую погрешность мы выбрали? Сформулируйте вывод в 2–3 предложениях.

Задача 3. Функция `relative_error`.

Напишите функцию:

```
def relative_error(approx, exact):
    return abs(approx - exact) / abs(exact)
```

Добавьте защиту: если `exact == 0`, вернуть `float('inf')` и не падать с `ZeroDivisionError`. Проверьте на четырёх парах из задачи 1.

Задача 4. Обратная задача — оценка точного значения.

Известно, что приближение $x = 47.3$ имеет относительную погрешность не более 0.1%. В каком диапазоне лежит x^*? Запишите ответ в виде интервала $[a, b]$ с разумным числом значащих цифр.

Задача 5. Порог по абсолютной и относительной ошибке.

Есть два датчика температуры. Датчик A врёт с абсолютной ошибкой не более 0.3 °C, датчик B — с относительной ошибкой не более 0.5%. При какой температуре датчик B точнее (в смысле $\Delta \le 0.3$)? Решите неравенство $|T| \cdot 0.005 \le 0.3$ и объясните, почему выбор датчика зависит от диапазона измерений.

Блок 2. Значащие цифры и округление (задачи 6–9)

Задача 6. Подсчёт значащих цифр.

Для каждого числа определите количество значащих цифр и запишите в экспоненциальной форме так, чтобы количество значащих цифр было однозначным:

1. 0.004520
2. 1200
3. 1.200
4. 10.03
5. 0.03050
6. 250000

Вывод: в каких случаях запись без экспоненциальной формы неоднозначна и почему?

Задача 7. Честная запись результата.

Измерение дало $x = 4.738291$ с абсолютной погрешностью $\Delta = 0.02$. Запишите результат в виде $x \pm \Delta$, оставив только верные значащие цифры (правило: последняя оставляемая цифра должна быть в том же разряде, что и погрешность). Сколько значащих цифр осталось? Что было бы неверно в записи 4.738291 ± 0.02?

Задача 8. Округление и banker's rounding.

Проверьте в Python:

```
print(round(0.5), round(1.5), round(2.5), round(3.5),
      round(-0.5), round(-1.5), round(-2.5))
```

Объясните, почему результат отличается от «школьного» правила «0.5 всегда вверх». Посчитайте, какой систематический сдвиг накопится на массиве из 10^6 чисел, каждое из которых — случайное $k + 0.5$ для $k = 0..9$, если округлять «вверх» вместо banker's rounding. Сделайте вывод, почему в финансах это критично.

Задача 9. Промежуточное округление — зло.

Вычислите двумя способами $S = \Sigma\ (k = 1..100)\ 1/k$:

- (а) без промежуточных округлений (float);
- (б) с округлением каждого слагаемого до 3 знаков после запятой.

Сравните с эталоном `math.fsum` и посчитайте ΔS и δS для обоих вариантов. Во сколько раз ошибка варианта (б) больше? Сформулируйте правило.

Блок 3. Накопление ошибки и float-эксперименты (задачи 10–15)

Задача 10. Классика 0.1 + 0.2.

Выведите `0.1 + 0.2`, сравните с `0.3` через `==` и через `math.isclose`. Найдите Δ и δ этого расхождения. Объясните в 3 предложениях (без формул, «на пальцах»), почему 0.1 не представимо точно в двоичной системе.

Задача 11. Катастрофическая потеря точности.

Вычислите $a - b$ для $a = 1 + 10^{-15}$, $b = 1$. Ожидаемый ответ — 10^{-15}. Сравните с фактическим результатом в Python. Посчитайте относительную погрешность результата. Затем перепишите квадратичную формулу для корней $ax^2 + bx + c = 0$ при $a = 1$, $b = 10^8$, $c = 1$ так, чтобы избежать вычитания близких чисел, и покажите, насколько точнее получается меньший корень.

Задача 12. Накопление в цикле — сравнение стратегий.

Сложите массив из 10^6 элементов `[0.1] * 1_000_000` тремя способами:

1. наивный цикл `for x in arr: total += x`;
2. `sum(arr)`;
3. `math.fsum(arr)`.

Сравните с эталоном 1e5. Для каждого способа посчитайте абсолютную и относительную погрешность. Постройте таблицу и сделайте вывод: какой способ и во сколько раз точнее.

Задача 13. Суммирование в правильном порядке.

Возьмите массив из 10^6 чисел: половина — очень маленькие (1e-10), половина — большие (1e6). Посчитайте сумму в порядке:

- (а) сначала маленькие, потом большие;
- (б) сначала большие, потом маленькие.

Сравните с `math.fsum`. Объясните, почему порядок влияет на точность и какая стратегия лучше (подсказка: «маленькие числа теряются, когда прибавляются к большим»).

Задача 14. Детектор накопления погрешности.

Реализуйте «эксперимент с ростом»:

```
x = 1.0
for _ in range(10_000):
    x *= 1.0001
```

Сравните с эталоном `math.exp(10000 * math.log(1.0001))`. Посчитайте δx итерационно — на каком шаге относительная погрешность превысит 10^{-9}, 10^{-6}, 10^{-3}? Постройте график δx от номера шага (matplotlib). Сделайте вывод: как быстро «раздувается» ошибка в мультипликативных процессах.

Задача 15. Финансы: float vs Decimal.

Смоделируйте начисление 0.01 рубля на счёт 10^6 раз подряд (это ~10 000 рублей):

- (а) в `float`;
- (б) в `Decimal` с точностью 28 знаков.

Сравните результаты и посчитайте расхождение в рублях и в процентах. Какой результат «правильный» с точки зрения бухгалтерии? Дополнительно реализуйте учёт в целых копейках (int) и покажите, что он точнее `float`. Сделайте вывод: почему в финансовых системах не используют `float`.

Бонус (по желанию)

Задача 16. Мини-исследование.

Сгенерируйте 10^5 случайных float в диапазоне [0, 1) и посчитайте среднее тремя способами (цикл, `sum`, `math.fsum`) и через `numpy.mean`. Сравните расхождения между ними в единицах $\varepsilon_{\text{machine}} \approx 2.2 \cdot 10^{-16}$. Оформите короткий отчёт: какой метод точнее и почему `numpy` обычно хорош, но не всегда равен `fsum`.

Главное, что проверяется: не «получить правильное число», а уметь объяснить, почему оно такое и насколько ему можно верить.